

Relazione Tecnica di Progetto: FlyToken

Sistema Full-Stack di Biglietteria Aerea con Certificazione Digitale su Blockchain Algorand

Progetto a cura di: Stirparo Noemi, Lomastro Alessandra

1. Introduzione e Scopo del Progetto

I sistemi di biglietteria convenzionali presentano limiti legati alla centralizzazione dei dati, al rischio di duplicazione dei titoli di viaggio e alla difficoltà per il passeggero di verificare in modo indipendente la validità del proprio titolo.

FlyToken è un'applicazione web full-stack sviluppata per la gestione e la prenotazione di voli aerei. L'obiettivo principale del progetto è simulare un sistema di biglietteria aerea che integri le tecnologie web tradizionali con la tecnologia **Blockchain**. All'atto dell'acquisto, ogni biglietto viene contestualmente emesso come **NFT (Non-Fungible Token)** standardizzato (ASA - Algorand Standard Asset) sulla rete pubblica di testnet di **Algorand**. L'hash univoco della transazione (*TxID*) funge da certificato crittografico immutabile associato alla prenotazione salvata nel database relazionale.

2. Specifica dei Requisiti

Il sistema è stato progettato per soddisfare le esigenze di due tipologie di utenti: **Clienti** (viaggiatori) e **Amministratori** (gestori della compagnia aerea).

2.1 Requisiti Funzionali

Lato Cliente (Accesso Pubblico):

- **Visualizzazione Voli:** L'utente può consultare in tempo reale l'orario e la disponibilità delle tratte aeree con dettagli su partenza, destinazione, orario, prezzo e posti disponibili.
- **Selezione Volo:** L'utente può selezionare il volo desiderato, specificare i dati del passeggero e la quantità di biglietti desiderati
- **Prenotazione e Acquisto:** L'utente può inserire il proprio nome, la quantità di biglietti desiderata, concludere la transazione e procedere all'acquisto.
- **Emissione Biglietto (Blockchain):** Al termine dell'acquisto, il sistema genera una ricevuta che include un ID univoco della prenotazione e il codice identificativo dell'NFT generato sulla rete Algorand, che funge da certificato di autenticità del biglietto.

Lato Amministratore (Accesso Protetto):

- **Autenticazione:** Accesso protetto da password per visualizzare i controlli avanzati.
- **Aggiunta Voli:** L'amministratore può inserire un nuovo volo nel database specificando partenza, destinazione, data, orario, prezzo e disponibilità posti.
- **Modifica Voli:** L'amministratore può aggiornare i dettagli di un volo già esistente.
- **Visualizzazione Storico:** L'amministratore può consultare lo storico di tutte le prenotazioni effettuate, inclusi i dettagli dei passeggeri, il totale incassato e gli ID degli NFT (*TxID*) associati ai biglietti.

2.2 Requisiti Non Funzionali

- **Persistenza dei Dati:** Le informazioni sui voli e sulle prenotazioni devono essere memorizzate in modo duraturo in un database relazionale.

- **Immutabilità e Tracciabilità (Blockchain P2P):** Il sistema si appoggia alla rete distribuita di Algorand (consenso *Pure Proof of Stake*) per garantire la non falsificabilità del titolo di viaggio.
- **Integrità (Blockchain):** L'emissione dei biglietti deve sfruttare una rete P2P decentralizzata (Algorand) per garantire la tracciabilità e l'immutabilità del token associato al biglietto, impedendo la contraffazione.
- **Usabilità ed Esperienza Utente (SPA):** Interfaccia Single Page Application sviluppata in React, reattiva e con rendering condizionale dei componenti senza ricaricamenti di pagina.
- **Sicurezza del Data Layer:** Le interrogazioni al database devono essere protette da attacchi di tipo SQL Injection.
- **RNF-05 - Tolleranza ai Guasti (Fault Tolerance):** Meccanismo di fallback trasparente nel backend: qualora il nodo Algorand o il faucet remoto siano temporaneamente irraggiungibili, il sistema assegna un codice di riserva controllato (**ALGO-FALLBACK-...**), completando regolarmente la registrazione locale senza interrompere l'esperienza utente.

3. Progettazione del Sistema (Architettura)

Il sistema è stato progettato seguendo un'architettura a tre livelli (*Three-Tier Architecture*), integrata con chiamate a servizi Blockchain esterni.

3.1 Frontend (Presentation Layer)

L'interfaccia utente è sviluppata utilizzando la libreria **React.js**.

- Gestisce lo stato dinamico dell'applicazione (es. voli caricati, pannello admin, carrello e ricevuta) interamente lato client tramite gli Hook (**useState**, **useEffect**).
- Comunica in modo asincrono con il server Node.js tramite chiamate HTTP RESTful utilizzando l'API **fetch**.

3.2 Backend (Application Layer)

Il server applicativo è sviluppato in **Node.js**. Riceve le richieste dal Frontend, processa la logica di business e dialoga con il database. Espone le seguenti API:

- **GET /api/flights:** Recupera dal database la lista dei voli disponibili.
- **POST /api/purchase:** Gestisce il core logic dell'acquisto. Coordina in modo asincrono la decurtazione dei posti disponibili, il salvataggio della prenotazione e richiama i servizi per simulare la transazione economica e il minting dell'NFT.
- **POST e PUT /api/flights:** Rotte protette per operazioni CRUD (Create, Update) sui voli da parte dell'admin.
- **GET /api/reservations:** Rotta protetta per il recupero dello storico prenotazioni.

La logica è stata modularizzata delegando specifiche operazioni ai file **FlightService.js** (per la gestione logica della prenotazione) e **AlgorandService.js** (per l'interfacciamento con la rete P2P).

3.3 Database (Data Layer)

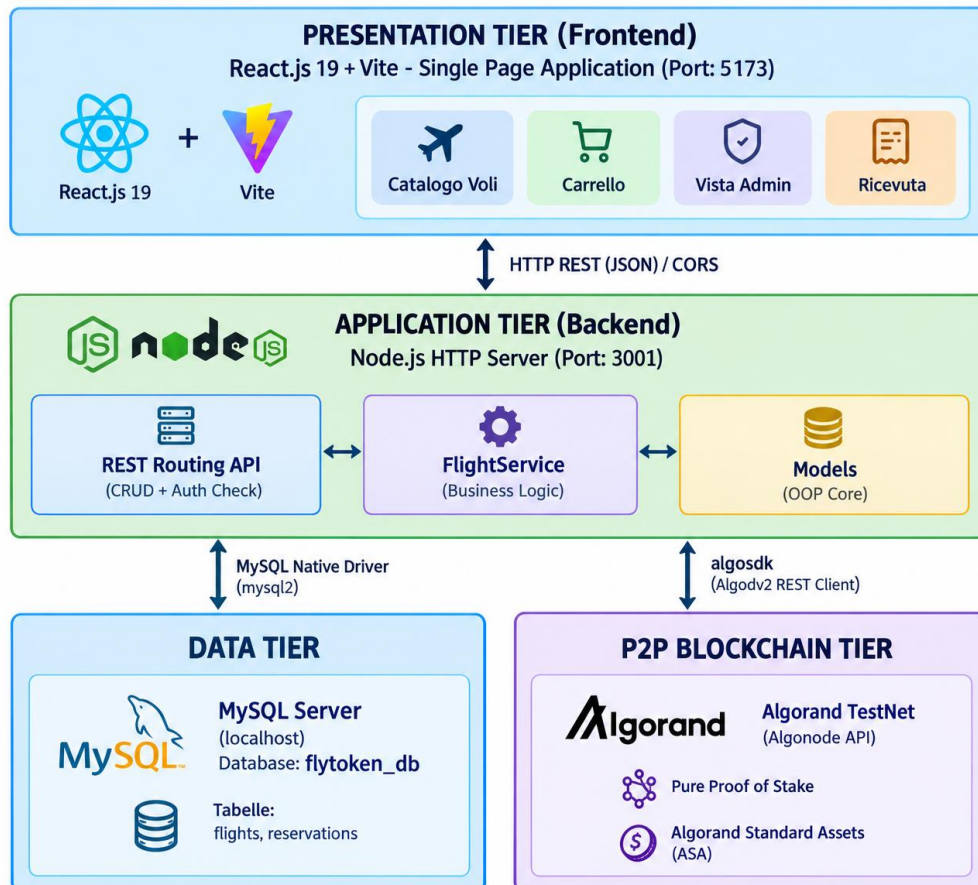
La persistenza locale è affidata al DBMS relazionale **MySQL**. Il database (**flytoken_db**) è normalizzato sulle entità fondamentali del dominio applicativo:

- **Tabella flights:** Memorizza i metadati dei voli.
- **Tabella reservations:** Traccia ogni singola prenotazione referenziando il volo acquistato e legando l'acquisto all'identificativo NFT.

Tutte le interazioni server-database (**INSERT**, **UPDATE**, **SELECT**) avvengono tramite libreria **mysql2** utilizzando query parametrizzate, garantendo la sanitizzazione degli input.

3.4 Livello Blockchain (Reti P2P)

Il modulo **AlgorandService** gestisce l'interfacciamento con la blockchain di Algorand. Durante un acquisto, il backend funge da client verso la rete P2P e richiede il "Minting" (la creazione) di uno Standard Asset (o NFT) che rappresenta digitalmente il biglietto. Una volta confermata la transazione dal network distribuito, il server riceve il Transaction ID (TxID) dell'asset creato e lo memorizza nel database locale per referenza incrociata.



3.5 Progettazione delle API RESTful

Il server **server.js** espone le seguenti interfacce:

Metodo	Endpoint	Ruolo	Descrizione	Autenticazione	Risposta
GET	/api/flights	Pubblico	Recupero elenco tratte disponibili	Nessuna	200 OK (JSON)
POST	/api/purchase	Pubblico	Minting NFT, salvataggio prenotazione, scalo posti	Nessuna	200 OK / 400 / 500
POST	/api/flights	Admin	Inserimento nuova tratta a catalogo	Header Authorization: Chiave	201 Created / 403
PUT	/api/flights/:id	Admin	Modifica dati tratta esistente	Header Authorization: Chiave	201 Created / 403
GET	/api/reservations	Admin	Storico prenotazioni e codici NFT	Header Authorization: Chiave	200 OK / 403

4. Testing e Validazione

La fase di testing è stata condotta tramite test manuali per verificare la corretta esecuzione dei flussi principali e la gestione degli errori. In particolare sono stati verificati i seguenti scenari:

1. **Flusso Amministratore:** È stato testato il blocco degli accessi non autorizzati. Inserendo la password corretta, è stato verificato l'inserimento di un nuovo volo nel database (con conseguente aggiornamento della UI) e la visualizzazione dello storico vendite.
2. **Flusso Cliente (Acquisto):** È stata simulata la selezione di un volo e l'inserimento dei dati del passeggero.
3. **Integrazione Blockchain:** Il test più critico ha riguardato la corretta comunicazione tra il server e la rete Algorand. È stato verificato che, a seguito di un acquisto, il server scali correttamente i posti dal database e riesca a effettuare con successo il Minting dell'NFT (Asset) sulla Testnet, restituendo il corretto Transaction ID visibile pubblicamente tramite Pera Explorer.

4. Conclusioni

FlyToken dimostra praticamente come l'unione tra lo sviluppo web full-stack tradizionale (React, Node, MySQL) e la tecnologia distribuita basata su Blockchain (Algorand) possa risolvere potenziali criticità del settore ticketing, offrendo tracciabilità verificabile pubblicamente, decentralizzazione della prova di acquisto e immutabilità del dato emesso.